PROBLEM STATEMENT 22

Stemroom

Online Music Collaboration Platform

Review 1: Architecture, Core Java, JDBC & UI Implementation



Portfolios, shared projects and deadlines for musicians.

TEAM DETAILS

ByteForce

Team Members:

- Om Singh
- Aryan Sharma
- Adi

1. Problem Understanding & Motivation

THE CORE PROBLEM STATEMENT

"Musicians collaborating remotely lack a dedicated, structured hub for their audio stems, portfolio, and project milestones—leaving files scattered across chat apps with no clarity on versions or deadlines."

Target Audience & User Pain Points

- Independent musicians, songwriters, and audio producers.
- Remote collaborators across different geographic zones.
- Audio stems and mix takes get buried in informal messaging apps.
- Confusion regarding master versions and pending deliverables.
- Lack of an integrated portfolio to showcase vetted original work.

The Stemroom Approach

- A focused, distraction-free web platform tailored to audio workflows.
- Explicit project management with deadlines and collaborator rosters.
- Quality gatekeeping: Content moderation as a core architectural step.
- Persistent musician portfolio with strict status transparency.
- Robust, zero-bloat stack built with native Servlets, JDBC and MySQL.

2. User Roles & Core Responsibilities

ROLE: MUSICIAN

Creative Portfolio & Collaboration

• Project Lifecycles: Create projects with title, description, and strict target deadlines.
• Team Collaboration: Invite fellow musicians by email; manage member permissions and status.
• Portfolio Management: Upload songs with lossless/compressed audio files (MP3, WAV, FLAC).
• Status Tracking: Immediate visibility into song moderation states (Pending, Approved, Rejected).
• Role Boundaries: Guarded by AuthFilter; strictly isolated from administrative endpoints.

ROLE: ADMINISTRATOR

Platform Governance & Quality Control

• User Administration: Provision accounts, toggle roles (Admin/Musician), and delete inactive users.
• Cascading Safety: User removal automatically and safely cascades to associated projects and songs.
• Song Content Moderation: Review submitted songs in a dedicated queue; approve or reject audio submissions.
• System Configuration: Toggle global public sign-ups on/off and enforce per-musician project limits.
• Analytics Dashboard: High-level overview of pending review queues and active user counts.

3. Scope: Review 1 Implementation vs Review 2 Roadmap

COMPLETED IN REVIEW 1 (WORKING NOW)

Foundation & Operational Modules

✔️ Full project directory structure and Maven WAR deployment lifecycle.
✔️ Relational database design (MySQL 8) with DDL, constraints & seed data.
✔️ JDBC database layer with PreparedStatement and custom CrudDao generics.
✔️ Session authentication & AuthFilter security boundary.
✔️ Musician project collaboration (create, list, add members, update status).
✔️ Song portfolio upload with multi-part disk storage.
✔️ Admin user management, content moderation, and dynamic settings.
✔️ Custom, responsive CSS design system with zero third-party frameworks.

PLANNED FOR REVIEW 2

Advanced Audio & Real-time Collaboration

⏳ In-browser HTML5 audio streaming & waveform visualizer.
⏳ Song editing workflow (metadata updates and audio file replacement).
⏳ Project-level song sharing with track stems distribution.
⏳ Time-stamped audio comments for precise musical feedback.
⏳ Real-time notifications center and activity audit log.
⏳ Enterprise auth enhancements (BCrypt/PBKDF2 hashing & CSRF tokens).

4. System Architecture & Request Lifecycle

1. Browser
HTTP Request
(GET / POST)
Form Data / Audio

2. AuthFilter
Session verification
Role check
Route protection

3. Servlet
Input validation
Orchestration
View selection

4. DAO Layer
PreparedStatement
RowMapper mapping
Connection pool

5. MySQL 8
Relational persistence
ACID integrity
Foreign key cascades

Return Pipeline: Servlet → JSP View → Escaped HTML Rendering

- JSP Isolation: Views reside strictly under /WEB-INF/views/ to prevent direct URL access from browsers.
- No Scriptlets: 100% pure JSTL (<c:forEach>, <c:if>, <c:out>) tags maintain separation of presentation from logic.
- Automatic XSS Prevention: All dynamic database text is output via <c:out value='...'/> to guarantee HTML escaping.
- Post/Redirect/Get Pattern: State-changing POST submissions redirect on success to prevent accidental duplicate submissions, while failures forward back with retained form inputs.
- No Heavy Framework Overhead: Built with clean Jakarta Servlet 6.0 and JDBC specifications to satisfy exact evaluation rubrics.

5. Database Schema & Relational Design Decisions

RELATIONAL SCHEMA ENTITIES

5 Core Normalized Tables in MySQL 8

- **users:**
  id (PK), name, email (UK), password_hash, role (ENUM)
- **projects:**
  id (PK), title, description, owner_id (FK), deadline, status (ENUM)
- **project_members:**
  project_id (PK, FK), user_id (PK, FK) [Join Table]
- **songs:**
  id (PK), owner_id (FK), title, album, file_name, status (ENUM)
- **settings:**
  setting_key (PK), setting_value (VARCHAR)

KEY ARCHITECTURAL DECISIONS

Data Integrity & Normalization

1. Single Owner Isolation

Project ownership lives directly on `projects.owner_id`. Collaborator join table only stores invited peers, avoiding duplicate owner entries.

2. Explicit Moderation States

Every uploaded track defaults to 'PENDING'. Only administrative approval unlocks full platform availability.

3. Dynamic Settings Store

Key-value schema in `settings` allows toggling registrations and limits without executing database schema migrations.

4. Cascading Foreign Keys

ON DELETE CASCADE guarantees that deleting an account completely purges associated projects and songs without orphaned rows.

5. Normalized Unique Emails

Emails stored in strict lowercase with UNIQUE constraint to prevent case-variation duplicate accounts.

6. Core Java & Secure JDBC Implementation

OOP & LANGUAGE FEATURES

Demonstrated Syllabus Concepts

• Inheritance: Abstract User base class extended by Admin and Musician subclasses.
• Polymorphism: Overridden canModerate() and getRole() called uniformly without type checks.
• Interfaces & Generics: CrudDao<T> implemented across UserDao, ProjectDao, and SongDao.
• Collections & Optionals: List<T> for multi-row queries; Optional<T> ensures null is never returned.
• Enums with Metadata: Role, ProjectStatus, and ModerationStatus with parse() and labels.
• Exception Wrapping: DataAccessException cleanly wraps checked SQLExceptions.
• Try-with-resources: Guarantees connections, statements, and result sets close safely.

SECURE JDBC CODE PATTERN

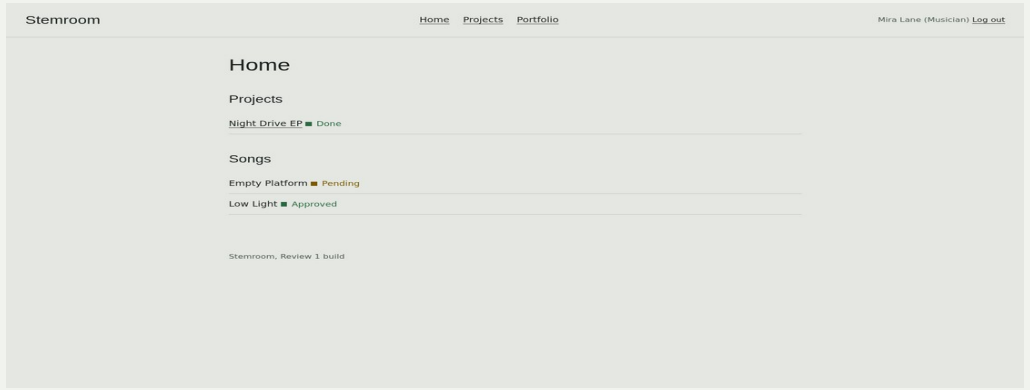
Parameterized Queries & RowMapper

```
// Example from SongDao.java
public Optional<Song> findById(int id) {
    String sql = """
        SELECT s.*, u.name AS owner_name
        FROM songs s
        JOIN users u ON s.owner_id = u.id
        WHERE s.id = ?
        """;
    return Db.queryOne(sql, this::mapRow, id);
}

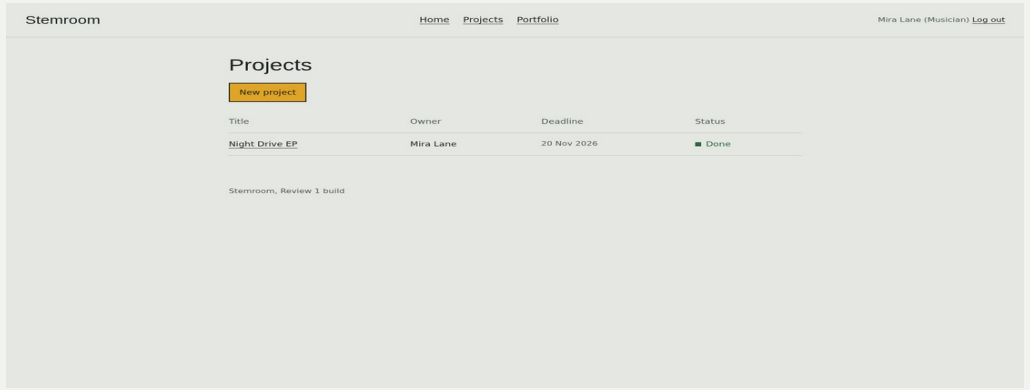
private Song mapRow(ResultSet rs) throws SQLException {
    return new Song(
        rs.getInt("id"),
        rs.getInt("owner_id"),
        rs.getString("title"),
        rs.getString("album"),
        rs.getString("file_name"),
        ModerationStatus.parse(rs.getString("status"))
    );
}
```

7. Live Application Interface & Visual Validation

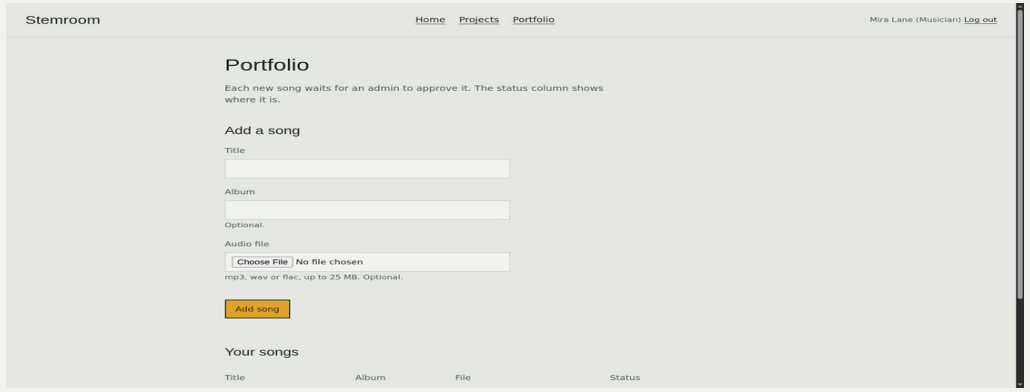
MUSICIAN DASHBOARD



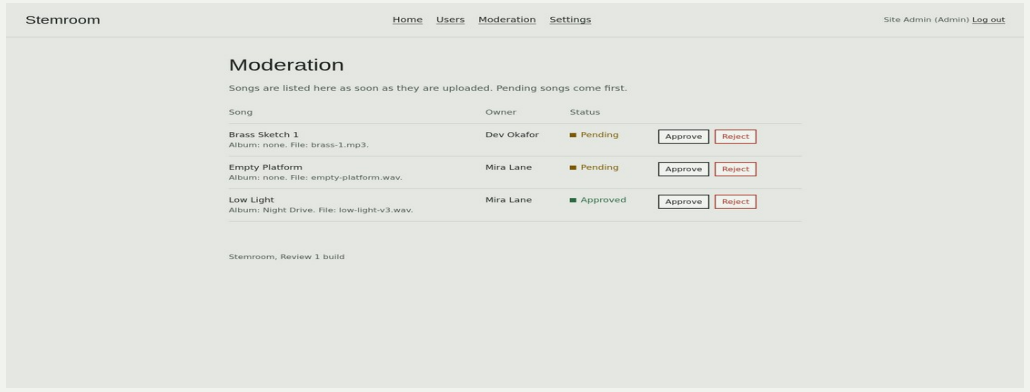
PROJECT COLLABORATION



PORTFOLIO & UPLOADS



ADMIN MODERATION QUEUE



8. Design System: Aesthetic Choices & Refused Cliches

PHILOSOPHY & PALETTE

"A rehearsal room with the lights on"

• Studio Canvas (#e5e8e2): Pale grey-green page background, soothing for late-night editing.
• Deep Ink (#1c2220): High-contrast green-black text and structural borders.
• Mustard Fill (#e0a526): Reserved exclusively for primary action buttons; never as decoration.
• Action Crimson (#a8362a): Used solely for destructive deletes and error boundaries.
• Approved Green (#2c6a43): Verification tags with paired square icons for full greyscale accessibility.
• Typography Pairing: Franklin Gothic condensed headings paired with legible, wide Verdana body text.

INTENTIONAL RESTRAINTS

What Was Deliberately Left Out

× No Generic AI Warm Cream & Terracotta:
Avoided cookie-cutter blog aesthetic.

× No Neon / Acid-Green on Pitch Black:
Avoided clichéd 'cyberpunk' developer tropes.

× No Rounded Drop-Shadow Cards:
Information layout follows editorial structure, not SaaS card grids.

× No Monospace Labels for Plain Data:
Keeps technical metadata natural to read for musicians.

× No Cluttered 'Welcome Back' Hero Banners:
Musicians immediately see actionable projects and tracks.

× Zero CSS Framework Bloat:
Handcrafted, lightweight CSS (~400 lines) with rapid sub-10ms paint times.

9. Review 1 Conclusion & Demonstration

REVIEW 1 STATUS

All Rubric Goals Satisfied

✔️ Problem Scope: Online Music Collaboration Platform (Problem Statement 22) fully addressed.
✔️ Database Architecture: MySQL 8 database normalized with cascading constraints & seed scripts.
✔️ Core Java Mastery: Inheritance, Polymorphism, Generics, Collections, and Custom Exceptions.
✔️ Web Layer: Jakarta Servlet 6.0, JSTL 3.0, and secure session AuthFilter.
✔️ Packaging & Deployment: Apache Tomcat 10.1 WAR deployment verified and running.
✔️ Open Source Delivery: Version controlled on GitHub (omsingh02/stemroom-byteforce).

PROJECT ROADMAP & DEMO

Ready for Evaluation

Live Demo Accounts Ready on http://localhost:8080/stemroom/:
• Admin: admin@stemroom.local / admin123
• Musician (Mira): mira@stemroom.local / musician123
• Musician (Dev): dev@stemroom.local / musician123

Repository:
https://github.com/omsingh02/stemroom-byteforce

Thank you!
Questions & Feedback Welcome.